



Classification III: Ensemble Models

Lecture 7

肖升生 博士

Dr. Shengsheng Xiao

Email: Xiao.shengsheng@shufe.edu.cn

- Ensemble Methods
 - Some Concepts
 - Bagging and Boosting
- Random Forest
 - Algorithm
 - Randomness
- Adaboost
 - Algorithm
 - Weight Update
- GBDT and XGBoost
- Case Study

- Ensemble Methods
 - Some concepts
 - Bagging and Boosting
- Random Forest
 - Algorithm
 - Randomness
- Adaboost
 - Algorithm
 - Weight Update
- GBDT and XGBoost
- Case Study

Classification

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training set



Learn classifier



Different Classifiers

Refund	Marital Status	Taxable Income	Cheat
No	Single	75K	?
Yes	Married	50K	?
No	Married	150K	?
Yes	Divorced	90K	?
No	Single	40K	?
No	Married	80K	?

Test set



Model



Predicted classes

Ensemble Methods

- Construct a set of classifiers from the training data
- Predict class label of previously unseen records by aggregating predictions made by multiple classifiers

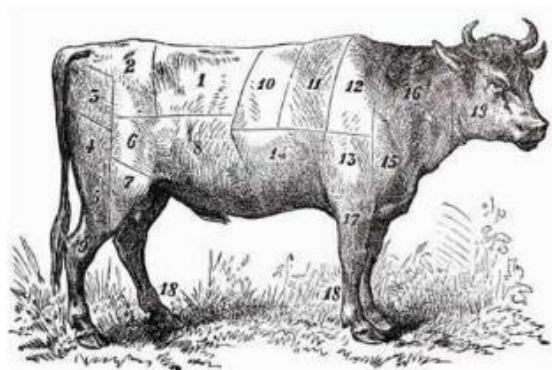
Why does it work?

- Suppose there are 25 base classifiers
 - Each classifier has error rate, $\varepsilon = 0.35$
 - Assume classifiers are independent
 - Probability that the ensemble classifier makes a wrong prediction:

$$\sum_{i=13}^{25} \binom{25}{i} \varepsilon^i (1 - \varepsilon)^{25-i} = 0.06$$

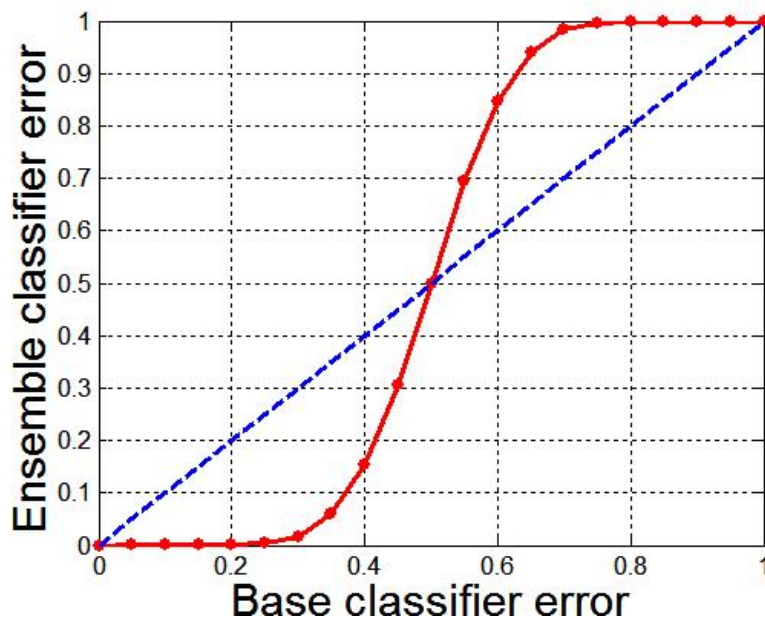
An Example

- The parable of the ox (Sir Francis Galton, 1906)
 - 787 people guessed the weight of an ox
 - Avg crowd guess: 1,197 pounds
 - True weight: 1,198 pounds



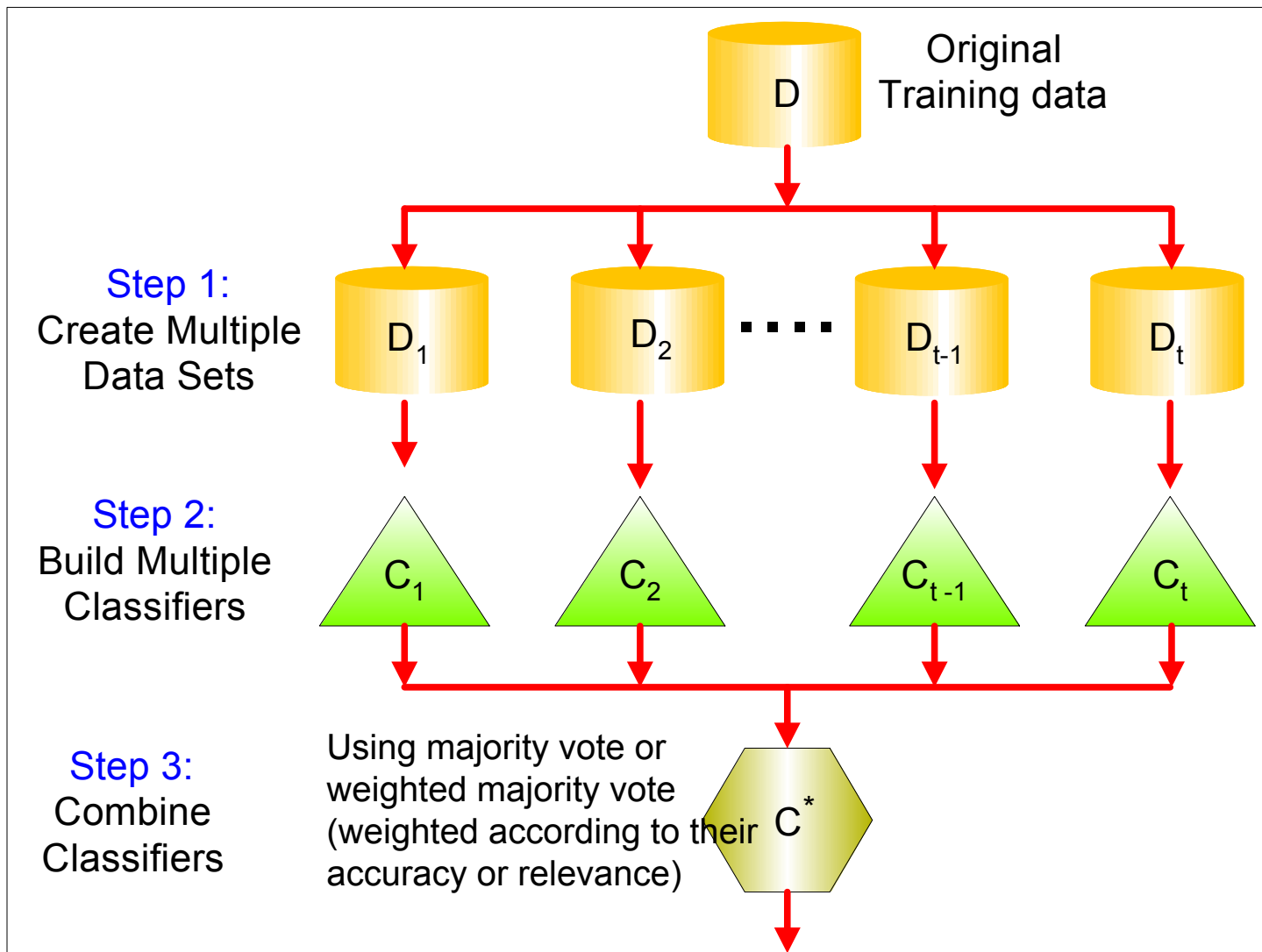
Necessary Conditions

- Ensemble Methods work better than a single base classifier if:
 1. All base classifiers are independent of each other
 2. All base classifiers perform better than random guessing (error rate < 0.5 for binary classification)



Classification error for an ensemble of 25 base classifiers, assuming their errors are uncorrelated.

General Idea



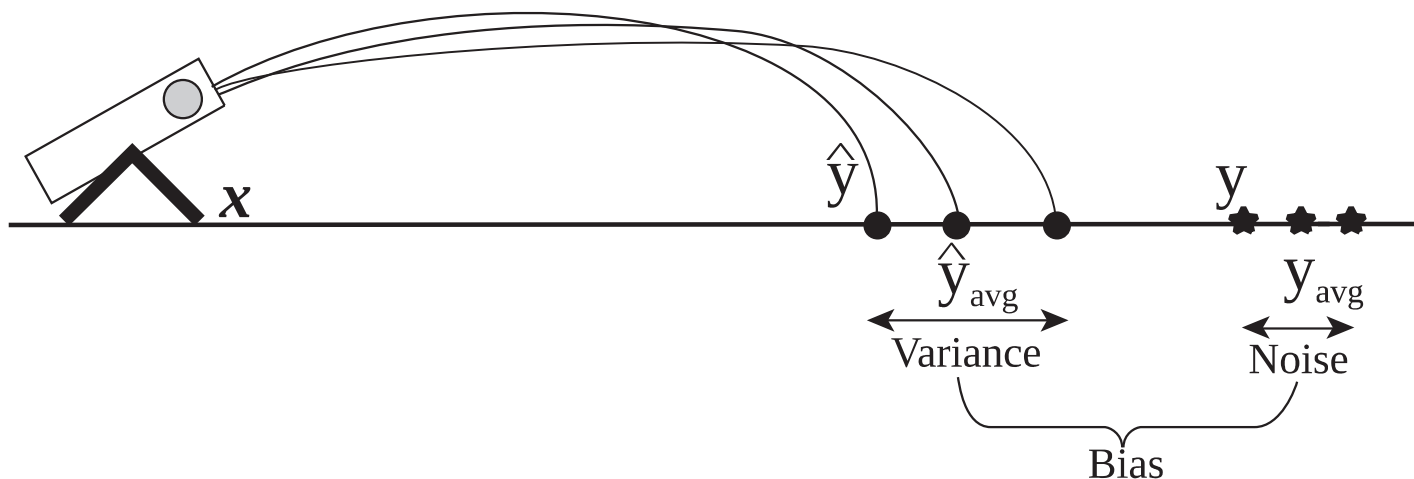


Ensemble methods

- Ensemble Methods work best with **unstable base classifiers**
 - Classifiers that are sensitive to minor perturbations in training set, due to *high model complexity*
 - Examples: Unpruned decision trees, ANNs, ...

Bias-Variance Decomposition

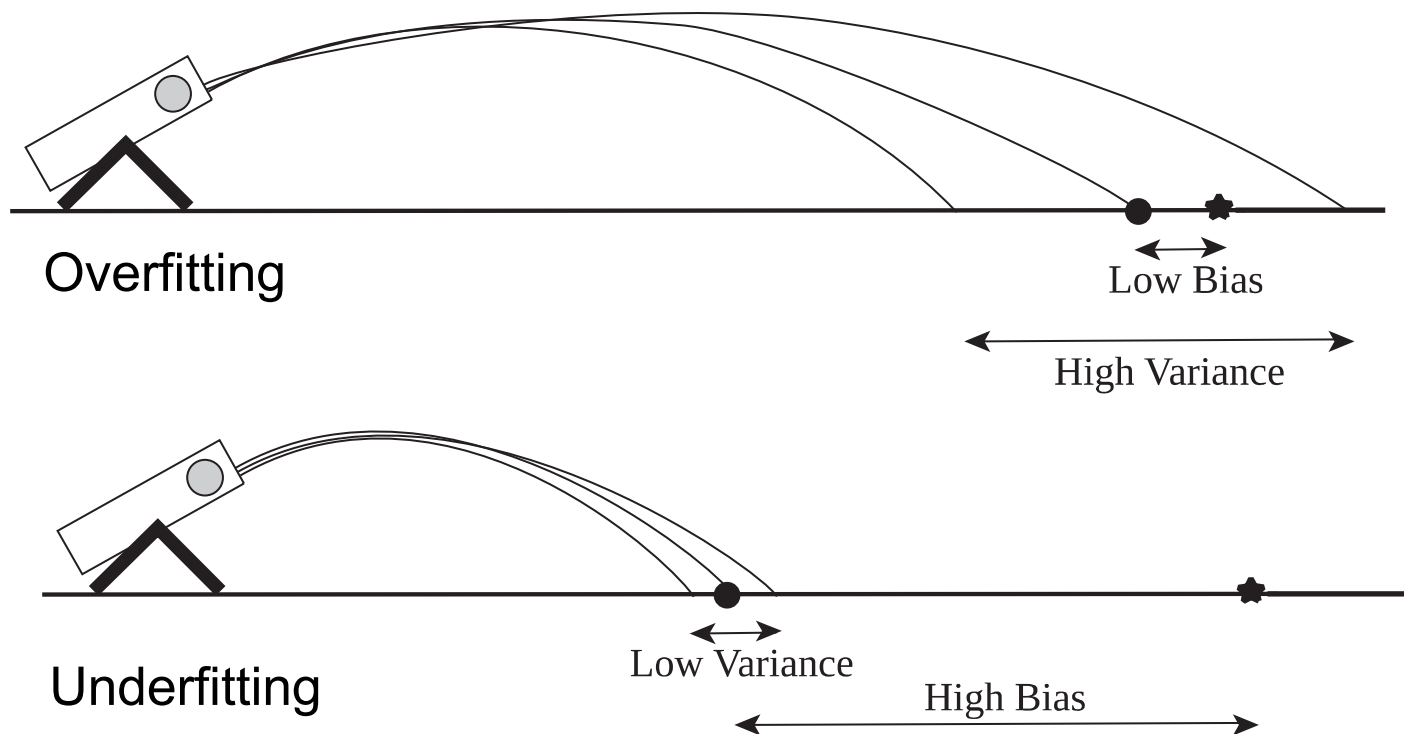
- Analogous problem of reaching a target y by firing projectiles from x (regression problem)



- For classification, gen. error of model m can be given by:

$$gen.error(m) = c_1 \times noise + bias(m) + c_2 \times variance(m)$$

Bias-Variance Trade-off and Overfitting



- Ensemble methods try to reduce the variance of complex models (with low bias) by *aggregating* responses of multiple base classifiers



Examples of Ensemble Methods

- How to generate an ensemble of classifiers?
 - **Bagging (Bootstrap Aggregating)**
i.e., Random Forest
 - **Boosting**
i.e., Adaboost、GBDT、XGBoost

- Ensemble Methods
 - Some concepts
 - Bagging and Boosting
- Random Forest
 - Algorithm
 - Randomness
- Adaboost
 - Algorithm
 - Weight Update
- GBDT and XGBoost
- Case Study

Bagging

- Sampling with replacement

Training Data
↙

Data ID	1	2	3	4	5	6	7	8	9	10
Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

- Build classifier on each bootstrap sample
- Each sample has probability $(1 - 1/n)^n$ of being selected as test data
- Training data = $1 - (1 - 1/n)^n$ of the original data

The 0.632 bootstrap

- This method is also called the *0.632 bootstrap*
 - A particular training data has a probability of $1-1/n$ of *not* being picked
 - Thus its probability of ending up in the test data (not selected) is:

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} = 0.368$$

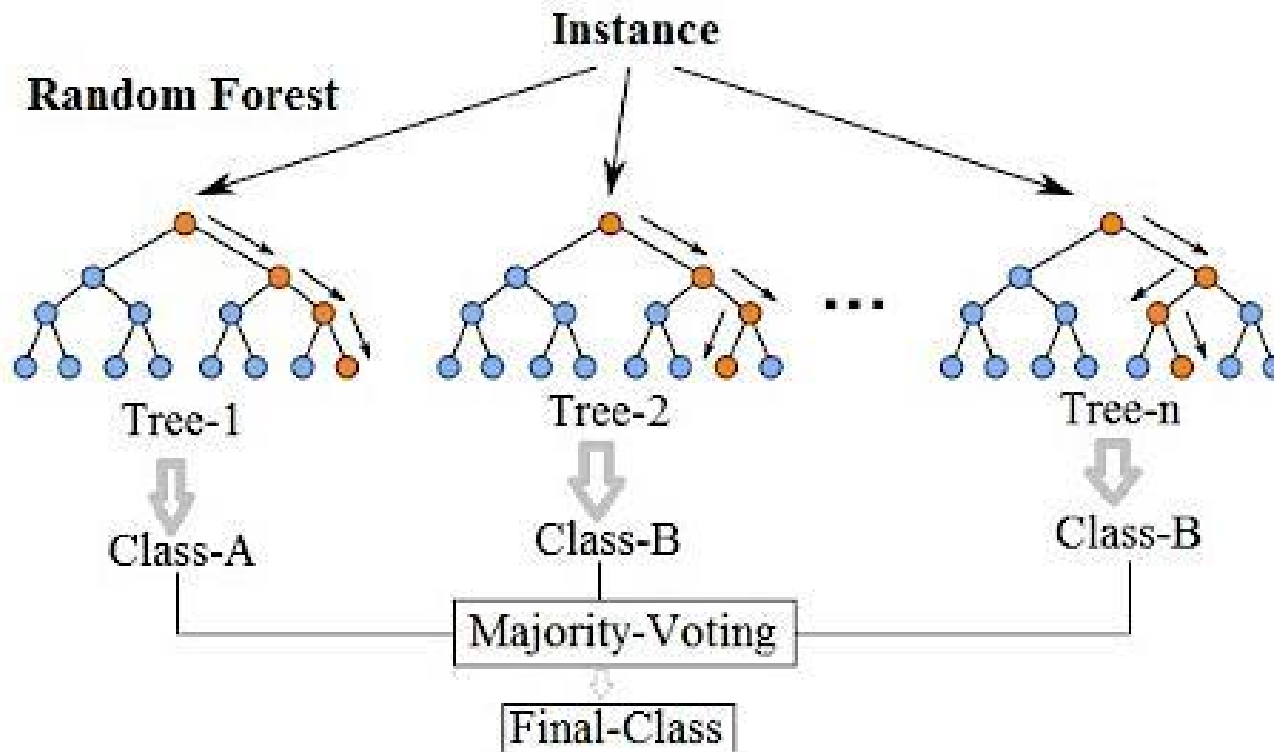
- This means the training data will contain approximately 63.2% of the instances

Bagging

- Bagging (Bootstrap aggregation):
 - Take B bootstrap samples from the training data, each of the same size as the training data.
 - Fit a large tree to each bootstrap sample. This will give us B trees.
 - Combine the results from each of the B trees to get an overall prediction (forest)

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

An Example



Disadvantages

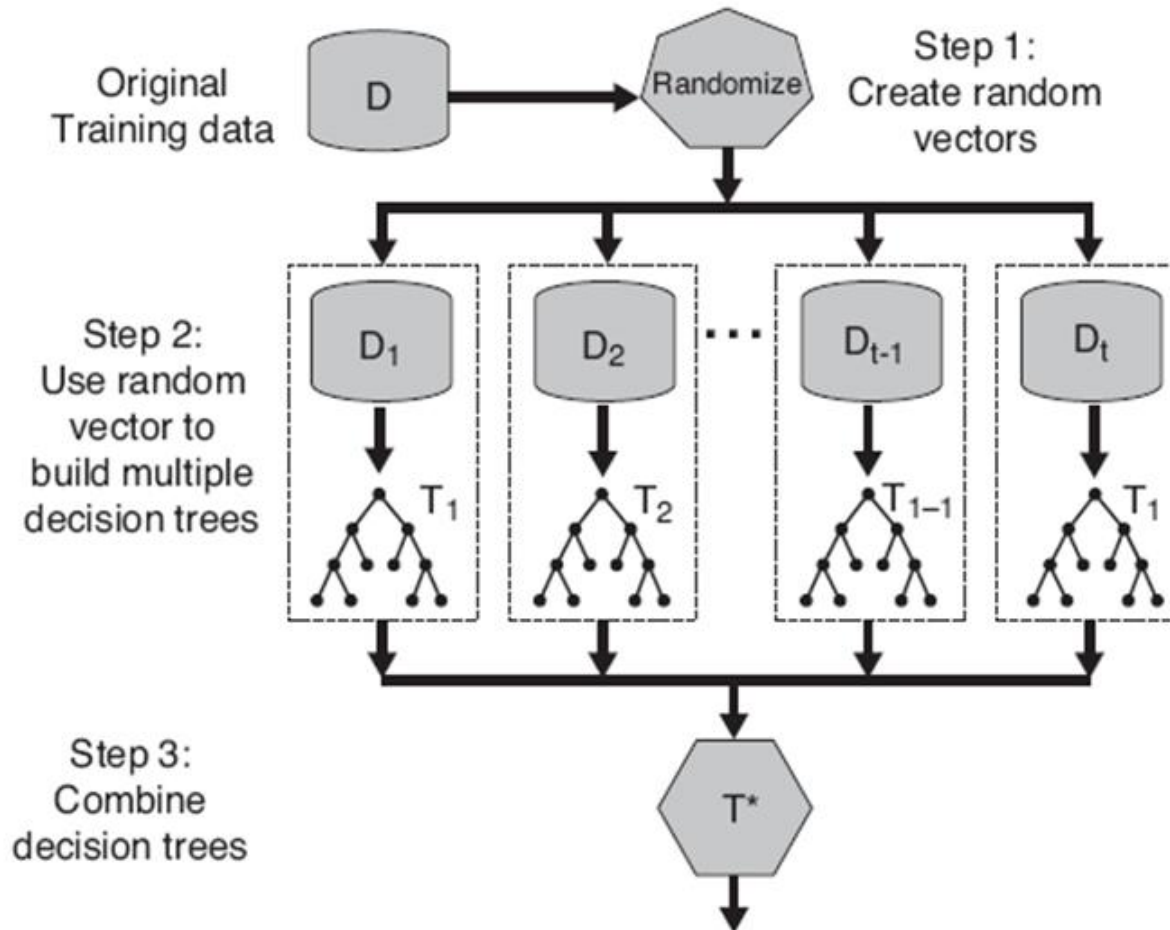
- Decision trees are greedy
 - They choose which variable to split on using a greedy algorithm that maximizes purity or information gain
 - Even with Bagging, the decision trees can have a lot of **structural similarities** and **correlation** in their predictions
 - But ensemble learning works best with **independent predictors**



Improvement: Random Forest

- Train a Bagged Decision Tree
- Use a modified tree learning algorithm that selects (at each candidate split) a random subset of features
 - If we have d features, consider $\sqrt{d}$ random features
- We can call this **Feature Bagging**
- It helps break the correlation between trees

Random Forest





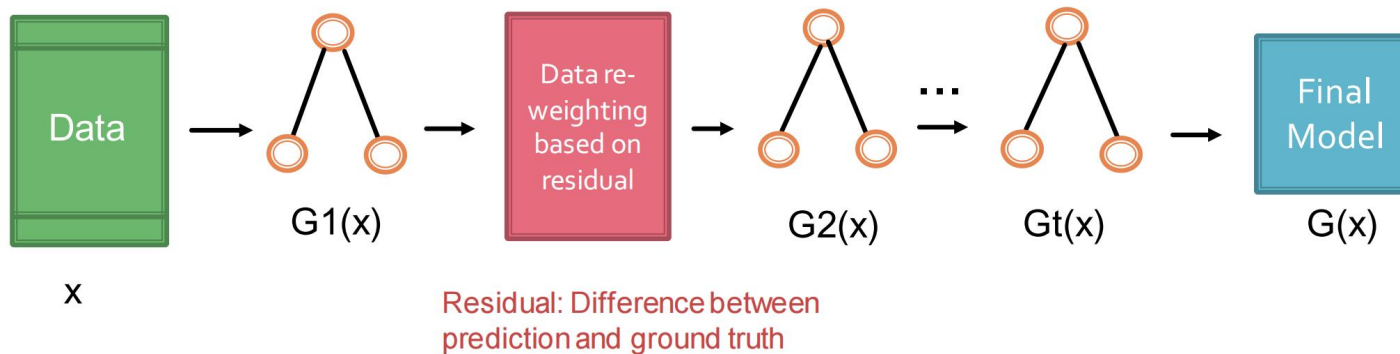
Random Forest Algorithm

- Construct an ensemble of decision trees by **manipulating training set as well as features**
 - Use bootstrap sample to train every decision tree (similar to Bagging)
 - Use the following tree induction algorithm:
 - At every internal node of decision tree, randomly sample p attributes for selecting split criterion
 - Repeat this procedure until all leaves are pure (unpruned tree)

- **Ensemble Methods**
 - Some concepts
 - Bagging and Boosting
- **Random Forest**
 - Algorithm
 - Randomness
- **Adaboost**
 - Algorithm
 - Weight Update
- GBDT and XGBoost
- Case Study

Boosting

- Another ensemble learning algorithm
 - Combines the outputs of many “weak” classifiers to produce a powerful “committee”
 - Learns multiple trees sequentially, each trying to improve upon its predecessor (**errors/residuals**)
 - Final classifier is weighted sum of the individual classifiers



Types of boosting

- **AdaBoost**
 - “Adaptive Boosting”
 - One of the originals
- **Gradient Boosting**
 - Uses gradient descent to create new learners
 - The loss function is differentiable
- **XGBoost**
 - “eXtreme Gradient Boosting”
 - Type of gradient boosting
 - Has become very popular in data science competitions

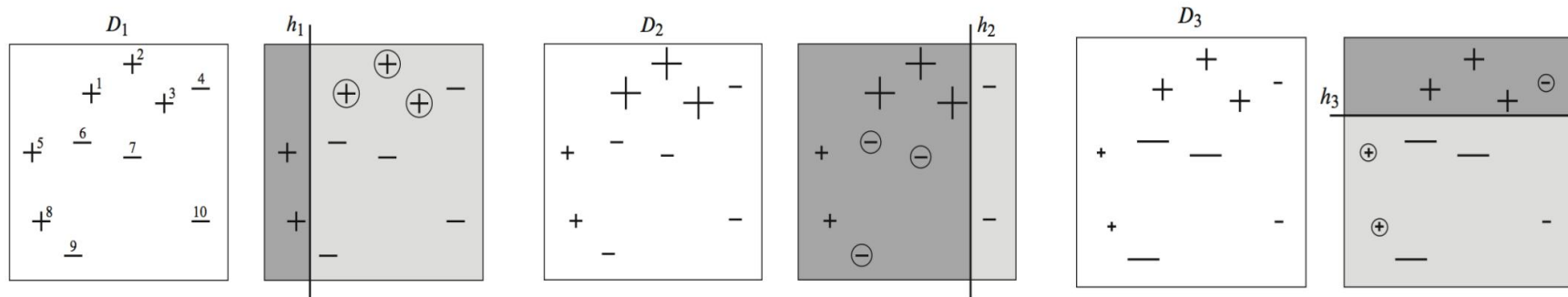
Adaboost

- An iterative procedure to adaptively change distribution of training data by focusing more on previously misclassified records
- Unlike bagging, weights may change at the end of a boosting round

Adaboost

- Equal weights are assigned to each training instance ($1/N$ for round 1) at first round
- After a classifier h_t is learned, the weights are adjusted to allow the subsequent classifier h_{t+1} to “pay more attention” to data that were misclassified by h_t .
- Final boosted classifier h^* combines the votes of each individual classifier

Adaboost: An Example



$$H(x) = \sum_t \rho_t h_t(x)$$

$$H = \text{sign} \left(0.42 \left(\begin{array}{|c|} \hline \text{shaded region} \\ \hline \end{array} \right) + 0.65 \left(\begin{array}{|c|} \hline \text{shaded region} \\ \hline \end{array} \right) + 0.92 \left(\begin{array}{|c|} \hline \text{shaded region} \\ \hline \end{array} \right) \right)$$

$$= \begin{array}{|c|} \hline \text{shaded region} \\ \hline \end{array}$$

Source: Schapire, R. E. and Freund, Y. (2012). Boosting: Foundations and Algorithms. MIT Press.

Adaboost

- **Input:** $(x_1, y_1), \dots, (x_m, y_m)$ where $x_i \in X, y_i \in \{-1, +1\}$
 - Training set D containing **N** instances
 - T rounds
 - A classification learning scheme
- **Output:**
 - A composite model

- Initialize equal weights for all observations
- At each iteration t :
 - 1. Train a classifier h_t using data weighted by D_{t-1}
 - 2. Compute the misclassification error adjusted by D_{t-1}
 - 3. Compute the weight of the current classifier α_t
 - 4. Reweight each observation based on prediction accuracy

Adaboost Algorithm

Input: $(x_1, y_1), \dots, (x_m, y_m)$ where $x_i \in X, y_i \in \{-1, +1\}$

Initialization: $D_1(i) = \frac{1}{m}, i = 1, \dots, m$

$D_t(i)$: probability distribution of x_i 's at time t

For $t = 1, \dots, T$:

- Find classifier $h_t : X \rightarrow \{-1, +1\}$ which minimizes error wrt D_t , i.e.,

$$h_t = \arg \min_{h_j} \varepsilon_j \text{ where } \varepsilon_j = \sum_{i=1}^m D_t(i)[y_i \neq h_j(x_i)]$$

minimize weighted error

- Weight classifier: $a_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$ for minimize exponential loss

- Update distribution: $D_{t+1}(i) = \frac{D_t(i) \exp[-a_t y_i h_t(x_i)]}{Z_t}$, Z_t is for normalization

Give error classified patterns more chance for learning.

Output: final classifier $\text{sign} \left(H(x) = \sum_{t=1}^T a_t h_t(x) \right)$

Weight update in Adaboost

- Assume: N training data in D , T rounds, (x_j, y_j) are the training data, h_t , a_t are the classifier and weight of the t -th round, respectively.
- Weight update on all training data in D :

$$D_j^{(t+1)} = \frac{D_j^{(t)}}{Z_t} \begin{cases} \exp^{-a_t} & \text{if } h_t(x_j) = y_j \\ \exp^{a_t} & \text{if } h_t(x_j) \neq y_j \end{cases}$$

where Z_t is the normalization factor

- Testing:
 - For each class c , sum the weights of each classifier that assigned class c to X (unseen data)
 - The class with the highest sum is the WINNER!

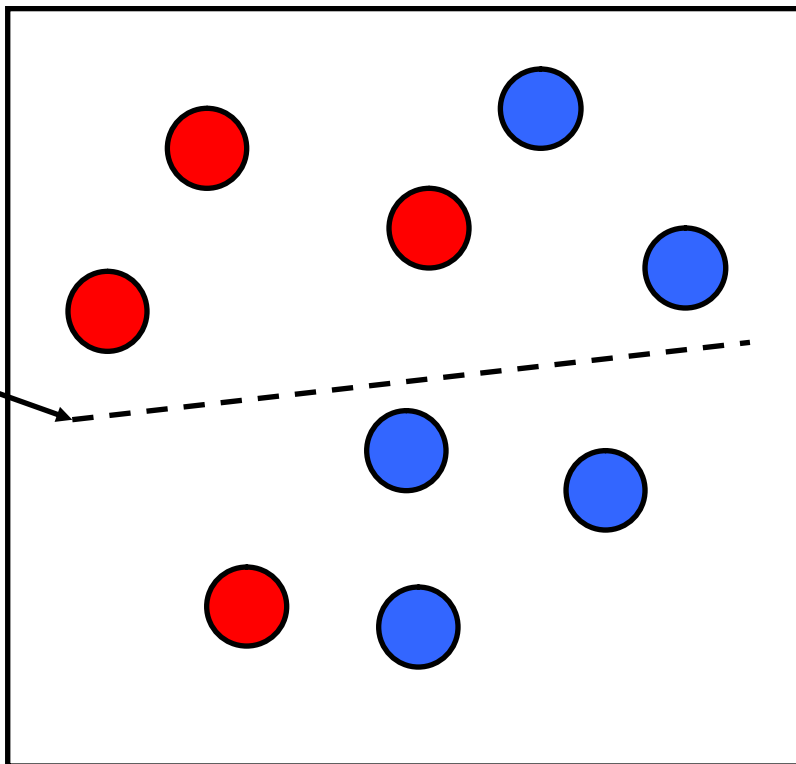
$$C_*(X_{test}) = \arg \max_y \sum_{t=1}^T a_t \delta(h_t(X_{test}) = y)$$

- Importance of a base classifier:

$$a_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

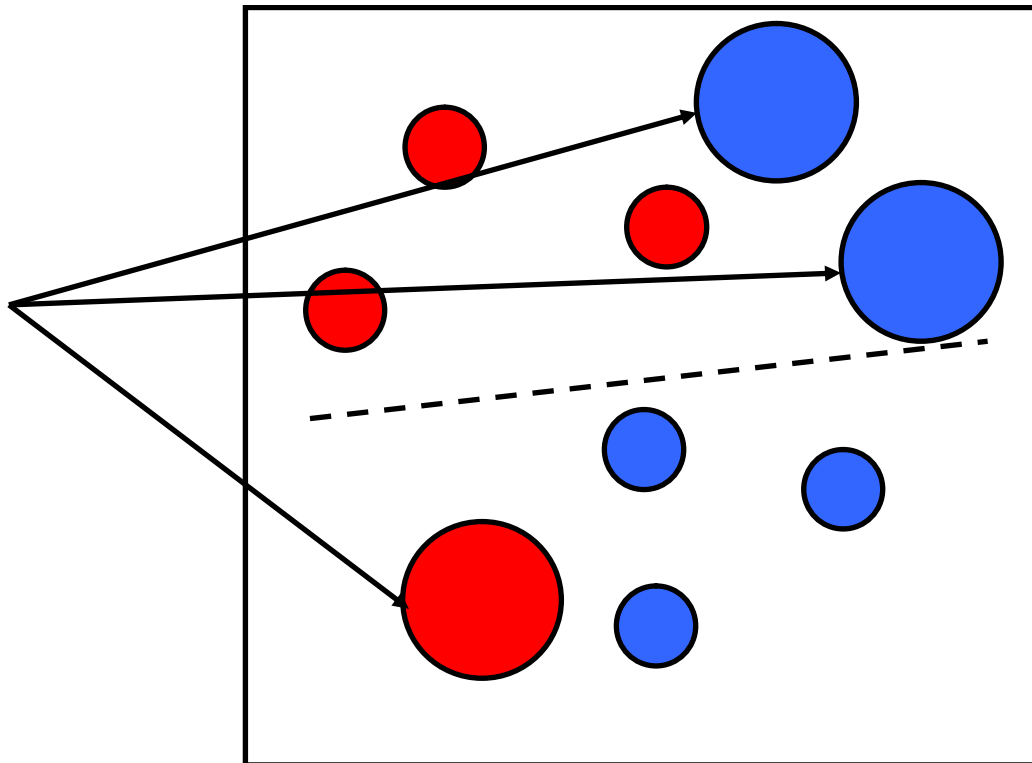
Adaboost Illustration

**Weak
Classifier 1**

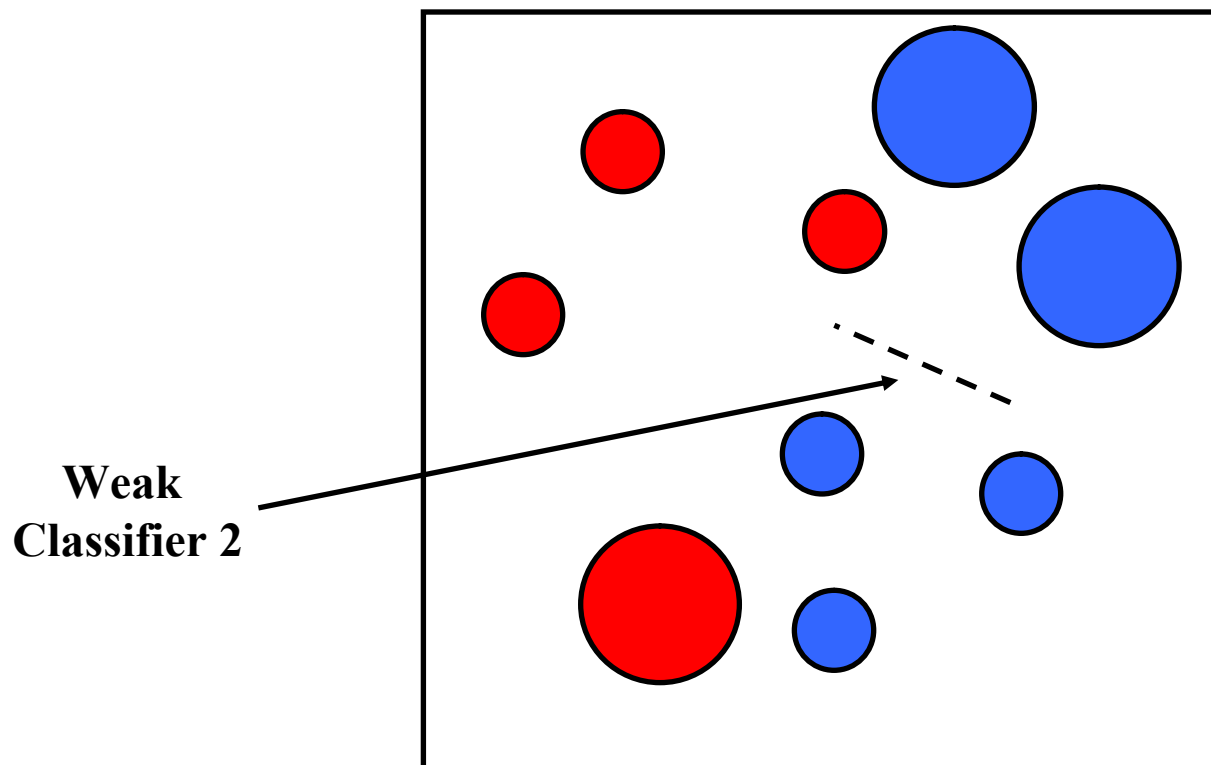


Adaboost Illustration

**Weights
Increased**

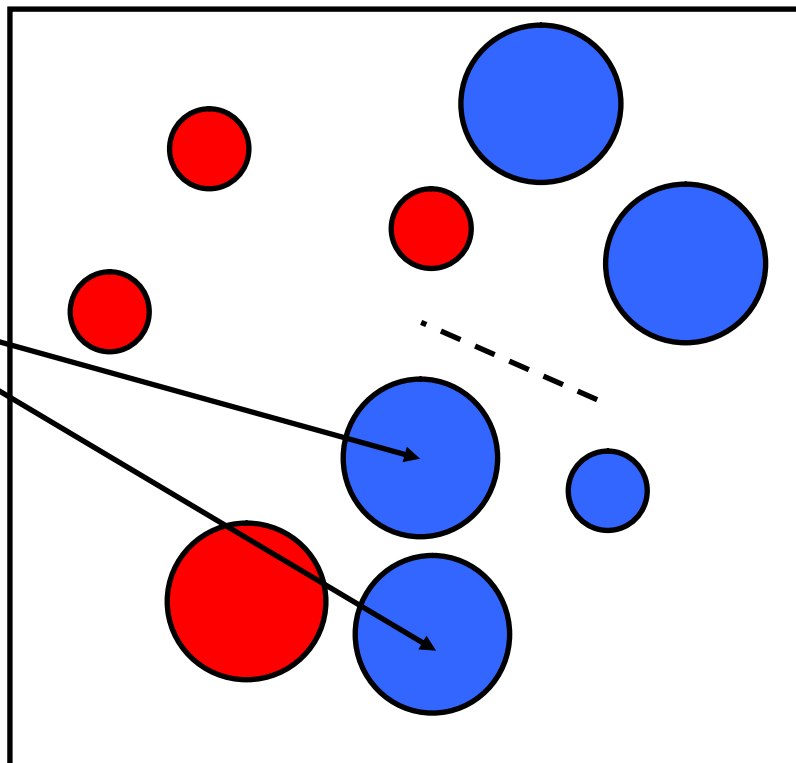


Adaboost Illustration

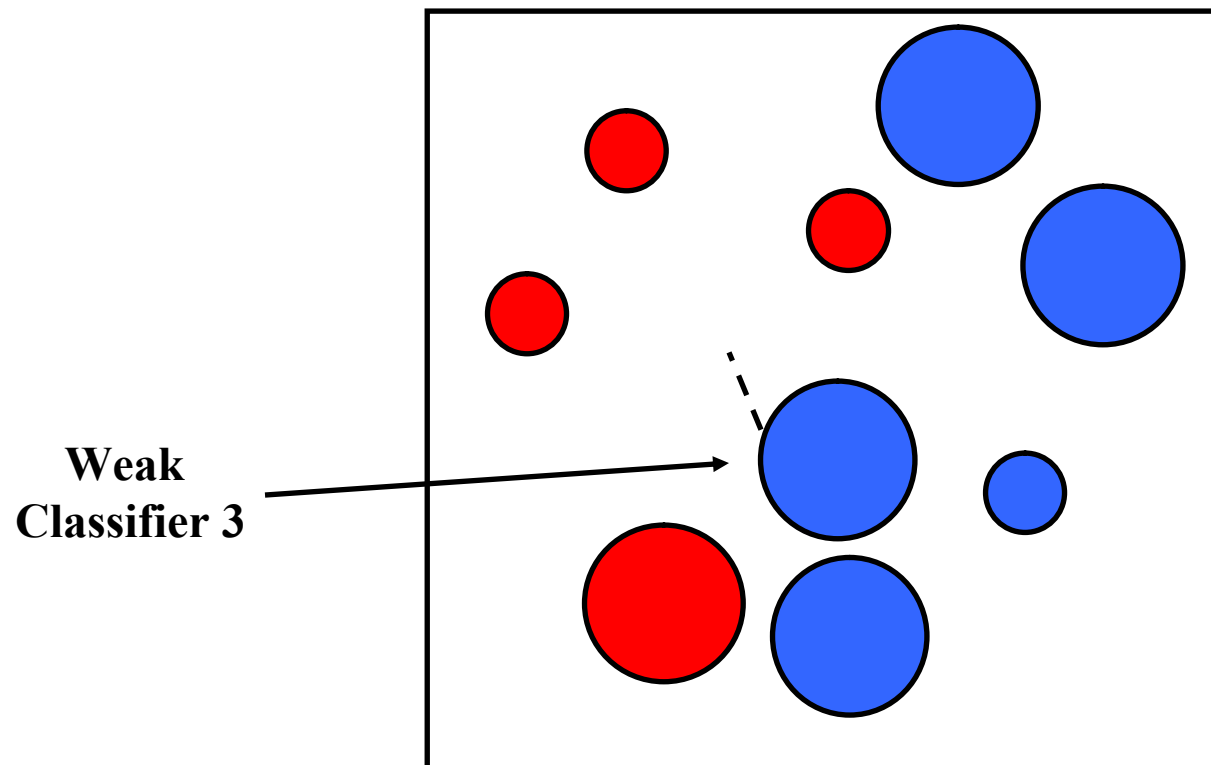


Adaboost Illustration

**Weights
Increased**

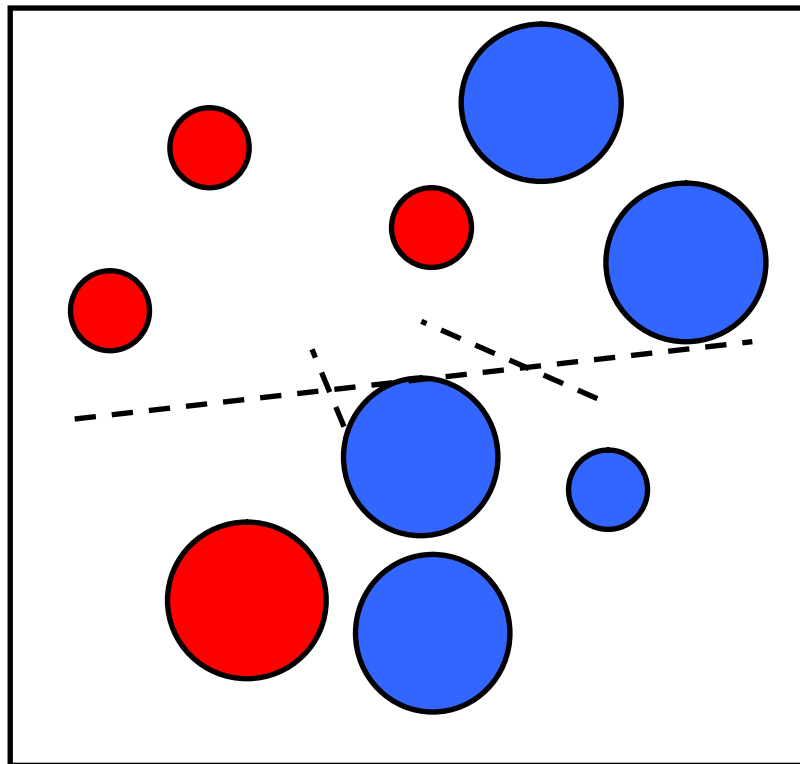


Adaboost Illustration



Adaboost Illustration

**Final classifier is
a combination of weak
classifiers**



- Final classifier in Adaboost:

$$\text{sign}\left(H(x) = \sum_{t=1}^T a_t h_t(x)\right)$$

- Equal to Minimize the following exponential loss function:

$$\text{loss}_{\text{exp}}[H(x)] = E_{x,y} \left[e^{-yH(x)} \right]$$

- **Ensemble Methods**
 - Some concepts
 - Bagging and Boosting
- **Random Forest**
 - Algorithm
 - Randomness
- **Adaboost**
 - Algorithm
 - Weight Update
- **GBDT and XGBoost**
- Case Study

- Gradient Boosting Decision Tree
- Optimize an Additive model
- Additive prediction model:

$$\hat{y}_i = \sum_{t=1}^T f_t(x_i) \quad f_t \text{ can be multi-level tree}$$

- Objective (cost) function:

$$\text{obj}(\theta) = \sum_{i=1}^N l(y_i, \hat{y}_i) + \sum_{t=1}^T \omega(f_t) \quad w(f_t) \text{ is a regularization term that models the complexity of the classifier}$$

- Algorithm

1. Fit a simple model $T^{(0)}$ on the training data

$$\{(x_1, y_1), \dots, (x_N, y_N)\}$$

Set $T \leftarrow T^{(0)}$. Compute the residuals $\{r_1, \dots, r_N\}$ for T .

2. Fit a simple model, $T^{(1)}$, to the current **residuals**, i.e. train using

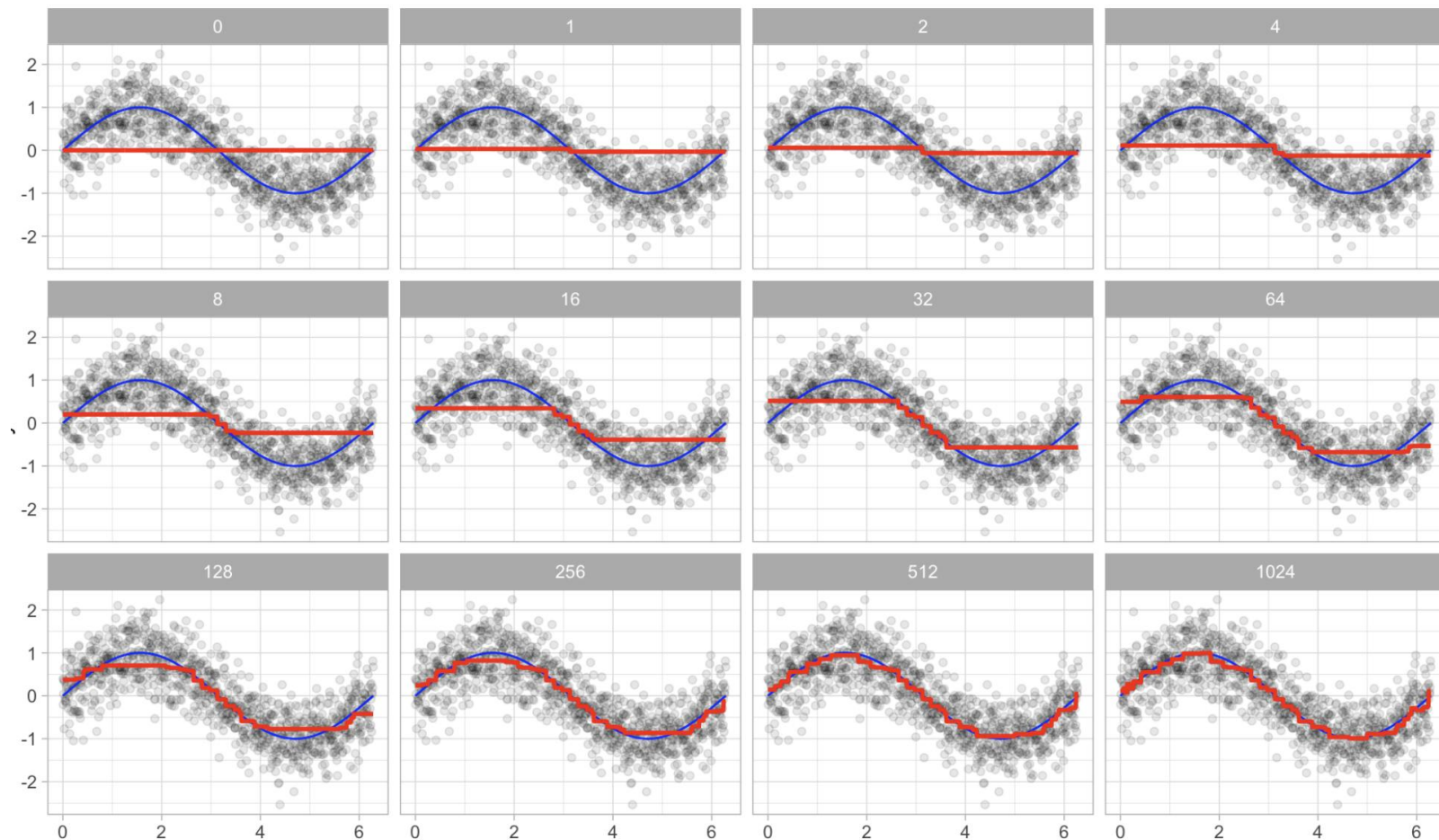
$$\{(x_1, r_1), \dots, (x_N, r_N)\}$$

3. Set $T \leftarrow T + \lambda T^{(1)}$

4. Compute residuals, set $r_n \leftarrow r_n - \lambda T^i(x_n)$, $n = 1, \dots, N$

5. Repeat steps 2-4 until **stopping** condition met.

GBDT: An Example



- How is this related to gradient descent?

Loss function $L(y, F(x)) = (y - F(x))^2/2$

We want to minimize $J = \sum_i L(y_i, F(x_i))$ by adjusting $F(x_1), F(x_2), \dots, F(x_n)$.

Notice that $F(x_1), F(x_2), \dots, F(x_n)$ are just some numbers. We can treat $F(x_i)$ as parameters and take derivatives

$$\frac{\partial J}{\partial F(x_i)} = \frac{\partial \sum_i L(y_i, F(x_i))}{\partial F(x_i)} = \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = F(x_i) - y_i$$

So we can interpret residuals as negative gradients.

$$y_i - F(x_i) = -\frac{\partial J}{\partial F(x_i)}$$

- How is this related to gradient descent?

$$F(x_i) := F(x_i) + h(x_i)$$

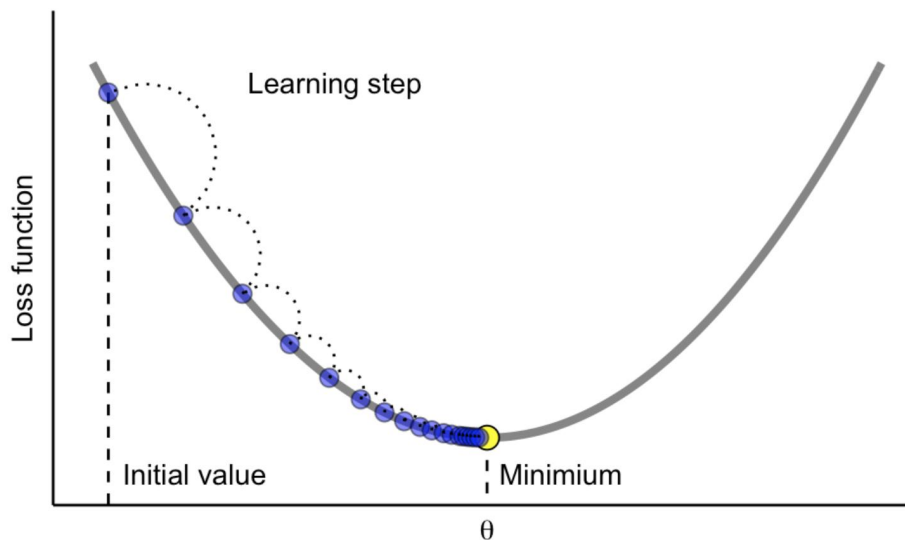
$$F(x_i) := F(x_i) + y_i - F(x_i)$$

$$F(x_i) := F(x_i) - 1 \frac{\partial J}{\partial F(x_i)}$$

$$\theta_i := \theta_i - \rho \frac{\partial J}{\partial \theta_i}$$

- GBDT algorithm can be generalized to other loss functions via gradient descent
- Update the model parameters in the direction of the loss function's descending gradient

$$\theta_i := \theta_i - \rho \frac{\partial J}{\partial \theta_i}$$



- XGBoost: eXtreme Gradient Boosting
 - A highly scalable implementation of gradient boosted decision trees with regularization
- System optimizations:
 - **Parallel tree constructions** using column block structure
 - **Distributed Computing** for training very large models using a cluster of machines
 - **Out-of-Core Computing** for very large datasets that don't fit into memory



Bagging v.s Boosting

- **Sample selection**
 - Bagging: selected with replacement; Boosting: No
- **Weight of each sample**
 - Bagging: same weight; Boosting: Dynamically changed
- **Result prediction**
 - Bagging: vote with equal weight;
 - Boosting: vote with different weight;
- **Parallel computing of each classifier**
 - Bagging: support parallel computing;
 - Boosting: do not support parallel computing

- Ensemble Methods
 - Some Concepts
 - Bagging and Boosting
- Random Forest
 - Algorithm
 - Randomness
- Adaboost
 - Algorithm
 - Weight Update
- GBDT and XGBoost
- Case Study

Case Study: Email Data

```
In [12]: from sklearn.ensemble import RandomForestClassifier
# Fitting a decision tree model with default parameters
rf = RandomForestClassifier(n_estimators = 100, random_state = 50)
rf.fit(X_train, y_train)
```

```
Out[12]: RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                                criterion='gini', max_depth=None, max_features='auto',
                                max_leaf_nodes=None, max_samples=None,
                                min_impurity_decrease=0.0, min_impurity_split=None,
                                min_samples_leaf=1, min_samples_split=2,
                                min_weight_fraction_leaf=0.0, n_estimators=100,
                                n_jobs=None, oob_score=False, random_state=50, verbose=0,
                                warm_start=False)
```

```
In [25]: from sklearn.ensemble import AdaBoostClassifier
adb = AdaBoostClassifier(n_estimators=100, random_state=50)
# Train Adaboost Classifier
adb.fit(X_train, y_train)
```

```
Out[25]: AdaBoostClassifier(algorithm='SAMME.R', base_estimator=None, learning_rate=1.0,
                             n_estimators=100, random_state=50)
```

```
In [39]: from sklearn.ensemble import GradientBoostingClassifier
# Train GBDT Classifier
gbc = GradientBoostingClassifier(loss='log_loss', learning_rate=0.1,
                                n_estimators=100, min_samples_split=2,
                                min_samples_leaf=1)
gbc.fit(X_train, y_train)
```

```
Out[39]: GradientBoostingClassifier
GradientBoostingClassifier()
```

Practice

id	age	sex	region	income	married	children	car	save_act	current_act	mortgage	pep
ID12101	48	FEMALE	INNER_CIT	17546	NO	1	NO	NO	NO	NO	YES
ID12102	40	MALE	TOWN	30085.1	YES	3	YES	NO	YES	YES	NO
ID12103	51	FEMALE	INNER_CIT	16575.4	YES	0	YES	YES	YES	NO	NO
ID12104	23	FEMALE	TOWN	20375.4	YES	3	NO	NO	YES	NO	NO
ID12105	57	FEMALE	RURAL	50576.3	YES	0	NO	YES	NO	NO	NO
ID12106	57	FEMALE	TOWN	37869.6	YES	2	NO	YES	YES	NO	YES
ID12107	22	MALE	RURAL	8877.07	NO	0	NO	NO	YES	NO	YES
ID12108	58	MALE	TOWN	24946.6	YES	0	YES	YES	YES	NO	NO
ID12109	37	FEMALE	SUBURBAN	25304.3	YES	2	YES	NO	NO	NO	NO
ID12110	54	MALE	TOWN	24212.1	YES	2	YES	YES	YES	NO	NO
ID12111	66	FEMALE	TOWN	59803.9	YES	0	NO	YES	YES	NO	NO
ID12112	52	FEMALE	INNER_CIT	26658.8	NO	0	YES	YES	YES	YES	NO
ID12113	44	FEMALE	TOWN	15735.8	YES	1	NO	YES	YES	YES	YES
ID12114	66	FEMALE	TOWN	55204.7	YES	1	YES	YES	YES	YES	YES
ID12115	36	MALE	RURAL	19474.6	YES	0	NO	YES	YES	YES	NO
ID12116	38	FEMALE	INNER_CIT	22342.1	YES	0	YES	YES	YES	YES	NO
ID12117	37	FEMALE	TOWN	17729.8	YES	2	NO	NO	NO	YES	NO
ID12118	46	FEMALE	SUBURBAN	41016	YES	0	NO	YES	NO	YES	NO
ID12119	62	FEMALE	INNER_CIT	26909.2	YES	0	NO	YES	NO	NO	YES
ID12120	31	MALE	TOWN	22522.8	YES	0	YES	YES	YES	NO	NO
ID12121	61	MALE	INNER_CIT	57880.7	YES	2	NO	YES	NO	NO	YES
ID12122	50	MALE	TOWN	16497.3	YES	2	NO	YES	YES	NO	NO
ID12123	54	MALE	INNER_CIT	38446.6	YES	0	NO	YES	YES	NO	NO
ID12124	27	FEMALE	TOWN	15538.8	NO	0	YES	YES	YES	YES	NO
ID12125	22	MALE	INNER_CIT	12640.3	NO	2	YES	YES	YES	NO	NO

Some Takeaways

- Ensemble Methods
 - Some concepts
 - Bagging and Boosting
- Random Forest
 - Algorithm
 - Randomness
- Adaboost
 - Algorithm
 - Weight Update
- GBDT and XGBoost



Acknowledgement

- Introduction to data mining (by Tan, Steinbach, Kumar)
- NYU: machine learning course (by Xi Chen)
- John Hopkins Course: Practical Machine Learning (by Jeff Leek, Roger Peng and Brian Caffo)

The End

■ THANK YOU

Any questions?

